

Resumo Completo — Linguagens Imperativas e Conceitos Relacionados

Este documento resume os principais conceitos sobre linguagens imperativas, incluindo tipagem, escopos, subprogramação, ponteiros, recursão e macros.

1. Tipagem das Linguagens Imperativas

1.1. Fracamente Tipadas

Permitem que variáveis de um tipo recebam valores de outro tipo e podem não possuir tipos de dados estritos. Oferecem mais liberdade, porém podem gerar:

- Desperdício de memória
- Erros de conversão
- Operações inválidas
- Arredondamentos indesejados

Exemplos: BASIC, C, PHP.

1.2. Fortemente Tipadas

São estritas quanto ao tipo das variáveis. Não permitem atribuições entre tipos diferentes sem conversão explícita. São mais seguras e menos ambíguas.

Exemplos: Java, Pascal.

2. Declaração de Variáveis e Escopo

2.1. Declaração

- Pode ser em uma seção fixa (ex: Pascal).
- Ou em qualquer ponto do código (ex: C, Java).

2.2. Tipos de Escopo

- **Global:** Existe durante todo o programa.
`int x;`
- **Local:** Existe apenas dentro de um subprograma.

```
void funcao() {  
    int y;  
}
```
- **De bloco:** Existe apenas dentro de um bloco { }.

- Variável z em { int z; }
- Variável i em loops: for(int i...)

Obs.: BASIC e Pascal (padrão) não possuem escopo de bloco.

Vantagens do escopo de bloco:

- Economiza memória.
- Facilita depuração.

3. Estruturas das Linguagens Imperativas

As LPs imperativas derivam do modelo de Von Neumann, baseado em iteração (loops) e desvios condicionais. Todas possuem:

- if/else
- for
- while

3.1. Flexibilidades em C

- **IF compacto (Operador Ternário):**
int z = x > 10 ? 0 : 1;
- **FOR extremamente flexível:** Permite omitir partes (inicialização, condição, incremento).
for(;;) { ... } // Loop infinito

3.2. While compacto (apenas C/C++)

Permite expressões múltiplas no próprio while:

```
while(puts("Digite"), scanf("%d", &x), x != 99) {
    ...
}
```

4. Subprogramação

4.1. Requisitos

- **Coesão máxima:** Faz apenas UMA tarefa.
- **Acoplamento mínimo:** Não deve depender de variáveis externas.

4.2. Formas de comunicação de subprogramas

1. **Variáveis globais (EVITAR):** Geram alto acoplamento.
2. **Entrada e saída (EVITAR):** Subprogramas não devem fazer I/O, exceto quando específicos para isso.

3. **Parâmetros por cópia:** Recebem uma cópia do valor.

```
int par(int x) { return !(x % 2); }
```

4. **Parâmetros por referência:** Recebem endereços de memória (necessitam de ponteiros em C).

```
void troca(int *x, int *y) {  
    int aux = *x;  
    *x = *y;  
    *y = aux;  
}
```

Nota: Constantes não podem ser passadas por referência.

Observação Importante

- **Linguagens com ponteiros explícitos:** C, C++, Pascal.
- **Sem ponteiros explícitos:** Prolog, Java, Python, Ruby, BASIC.

5. Ponteiros (C e Pascal)

Um ponteiro tem 3 fases:

1. **Declaração:**

```
int *p;
```

2. **Apontamento:**

```
p = &x;
```

3. **Acesso ao conteúdo:**

```
*p = 8; // Equivale a: x = 8;
```

6. Recursividade

É uma função que chama ela mesma.

Exemplo (Fatorial):

```
int fat(int n) {  
    if (n == 1) return 1;  
    else return n * fat(n - 1);  
}
```

7. Macros (Pré-processador do C)

Macros são substituições de texto feitas **ANTES** da compilação.

Exemplos:

```
#define MAX 100
#define dobro(x) 2*x
#define max(x,y) x>y?x:y
#define imprime(x) puts(x)
```

Problemas com macros

Podem causar erros porque não são funções, são substituições literais.

- **Exemplo que dá errado:**
#define dobro(x) 2*x
int a = dobro(2+3);
// A substituição vira: 2*2 + 3, que resulta em 7 (e não 10).
- **Correção (uso de parênteses):**
#define dobro(x) 2*(x)
// A substituição vira: 2*(2+3), que resulta em 10.

Resumo Final

- Linguagens imperativas podem ser **fortemente** ou **fracamente** tipadas.
- Escopo pode ser **global**, **local** ou de **bloco**; blocos só existem em algumas linguagens.
- Estruturas imperativas vêm do modelo de **Von Neumann**.
- **C** é a linguagem com maior flexibilidade de sintaxe (for, if, while).
- Subprogramas devem ter **alta coesão** e **baixo acoplamento**.
- Comunicação: por globais, I/O, parâmetros por cópia ou referência.
- Ponteiros possuem 3 fases: **declaração**, **apontamento** e **acesso**.
- **Recursividade** é uma função chamando ela mesma.
- **Macros** são substituições de texto, não funções; podem gerar erros se mal definidas (falta de parênteses).